

1. What strategies did you implement to ensure scalability and fault tolerance in backend services in projects like Skyplan at United Airlines?

Ans: In the Skyplan project at United Airlines, which handles critical flight planning and operations, scalability and fault tolerance were paramount considerations due to the high volume and mission-critical nature of data. To ensure scalability, I designed microservices leveraging Spring Boot that are stateless, enabling easy horizontal scaling through container orchestration platforms. Load balancing techniques were employed to distribute requests evenly among service instances.

I implemented asynchronous processing patterns where possible using messaging queues to decouple components and improve throughput.

For fault tolerance, circuit breakers and retry mechanisms were incorporated to gracefully handle transient failures, ensuring system resilience under partial outages.

Health checks and monitoring endpoints built into services allowed proactive detection of issues, integrated with observability tools for real-time alerts.

Database resiliency was addressed by applying replication and failover capabilities within PostgreSQL to prevent data loss and minimize downtime.

Lastly, extensive unit and integration testing with mocks and stubs ensured that failure scenarios were simulated and handled properly before production deployments.

These strategies combined resulted in backend services capable of handling dynamic workloads reliably while maintaining high availability in a complex airline operations ecosystem.

2. How do you apply secure coding practices in Java and Spring Boot applications?

Ans: In my experience developing with Java and Spring Boot, secure coding practices are paramount.

This involves validating and sanitizing all user inputs to prevent SQL injection and XSS attacks.

Utilizing the Spring Security framework has helped integrate authentication mechanisms effectively.

For instance, applying role-based access control ensures that users are granted the least privilege necessary to perform their tasks.

Also, regularly utilizing libraries for secure password storage using Bcrypt or PBKDF2 is essential for protecting sensitive data.

3. How do you approach security testing in your software development lifecycle?

Ans: My approach to security testing is integrated at every stage of the software development lifecycle (SDLC).

Starting from the requirements phase, I collaborate with architects to identify potential security threats associated with each design.

During the coding phase, I embrace practices like peer reviews and automated scans using tools like Sonar to identify vulnerabilities early.

In the testing phase, I run static and dynamic testing with tools such as Veracode and Postman for API assessments to ensure functionality and security compliance.

Finally, I ensure planned security assessments post-deployment, confirming vulnerability management protocols are actively maintained.

4. What tools and frameworks did you use to manage RESTful Web Services?

Ans: For managing RESTful Web Services, I primarily utilized Spring Boot due to its ease of use in setting up production-ready applications quickly.

I also integrated Maven as our build tool, which helped manage project dependencies efficiently.

In terms of API documentation, I employed Swagger, which allowed us to create interactive API documentation, making it easier for frontend developers and external stakeholders to understand and use our services.

Furthermore, CI/CD deployment was managed through Jenkins, facilitating automated testing and deployment processes to various environments.

5. Can you provide an example of how you handled version control using Git in your projects?

Ans: In my projects at Mphasis, notably during the FX0 application development, I utilized Git for version control. I followed a branching strategy that involved main, development, and feature branches to manage collaboration efficiently among team members. This approach allowed for isolated development, resulting in fewer conflicts and easier integration of new features. I ensured to regularly merge changes back into the develop branch, and utilized pull requests to discuss and review code before merging into main.

6. Hello BhaskerReddy T, How do you stay organized and manage your time between various tasks or projects?

Ans: As a seasoned IT professional with over 9 years of experience in managing complex software projects, I have developed a robust methodology for staying organized and managing my time effectively across various tasks. The fast-paced nature of the industry, especially when working on large-scale projects like the Skyplan flight planning software and the Security Remediation initiative at United Airlines, has required me to prioritize my workload and streamline my processes to ensure timely delivery and high-quality outcomes. One of the techniques I find invaluable is the use of Agile methodologies. Utilizing tools like JIRA enables me to break down projects into manageable sprints. For instance, during the development of the GOS Migration project, I coordinated with my team to establish clear goals for each sprint, which allowed us to focus on specific tasks without overwhelming ourselves. This also provided visibility into our progress, making it easier to recalibrate if we encountered any roadblocks.

To complement Agile practices, I employ a personal task management system that incorporates both digital tools and traditional methods. I often use IntelliJ Idea for coding tasks, but I also maintain a physical planner where I jot down daily and weekly goals. This hybrid approach helps me stay grounded, especially during intensive development cycles. For example, while working on the BillPay application at Mphasis, I had to juggle various responsibilities, from coding new features to deploying updates using Jenkins. By prioritizing tasks based on urgency and importance, I was able to maintain focus and meet project deadlines. Additionally, communication plays a vital role in my organization and time management. I ensure that I am in regular contact with my team members and stakeholders to align our objectives. During the Taco Bell project, effective communication allowed us to swiftly address client feedback, making necessary adjustments to our timelines and deliverables. I find that maintaining transparency not only builds trust but also fosters a collaborative atmosphere where everyone is encouraged to contribute their ideas.

Lastly, I am a firm believer in using time-blocking techniques to allocate specific periods in my calendar for focused work. By setting aside uninterrupted time to tackle complex coding tasks or to conduct code reviews, I minimize distractions and enhance productivity. For example, while migrating microservices to Java 17, I blocked out my afternoons for focused coding sessions, which significantly improved my workflow. This strategic approach has enabled me to handle multiple projects, ensure code quality, and contribute meaningfully to my team's success.

7. How do you ensure performance in high-load environments, especially in projects like FX0?

Ans: In high-load environments like FX0, ensuring database performance is crucial. I started by analyzing queries and transactions to identify potential bottlenecks. This included rewriting inefficient queries and eliminating unnecessary complexity.

Indexing is another vital component. For heavy read operations, I enforced strategic indexing on critical queries to speed up access times. Additionally, I utilized partitioning for large datasets to reduce query scanning times. Horizontal scaling was also implemented within these projects. By sharding databases based on certain criteria, I effectively distributed the load and improved availability and performance. Throughout, monitoring tools ensured that I could proactively address potential performance issues.

8. How have you utilized containerization tools like Docker in your API

deployments?

Ans: Utilizing containerization tools, especially Docker, was a cornerstone in my deployment strategies across various projects, including BillPay and Security Remediation. Using Docker allowed me to create isolated environments for debugging and testing APIs prior to full production.

For instance, during the deployment of microservices in Security Remediation, I encapsulated the services into Docker containers. This practice ensured that the environments for development, testing, and production were consistent, eliminating "it works on my machine" issues.

Moreover, Docker's orchestration features were instrumental in managing multiple microservices interacting together, allowing for easy scaling and management of the service ecosystem.

I also built Docker images tailored for specific microservices, streamlining the deployment cycle using CI/CD strategies with tools like Jenkins. This facilitated quicker rollouts and efficient version control.

Lastly, leveraging Docker enabled rapid iteration and automated rollback features in case of failures, significantly enhancing reliability and uptime for deployed APIs.

9. What strategies do you adopt to facilitate Agile sprint retrospectives in your teams?

Ans: Facilitating effective sprint retrospectives is crucial in an Agile environment for continuous improvement. During my experience with various projects such as Payment Connect and BillPay, I developed specific strategies to make these retrospectives constructive and insightful.

Initially, I ensured a safe space was maintained, encouraging open dialogue so that team members could express their thoughts freely without fear of judgment. I often started the retrospective by revisiting our sprint goal and recap achievements to highlight the team's hard work.

Incorporating structured formats like Start, Stop, Continue helped guide the discussion effectively. Team members would share what practices they found beneficial, what should be halted, and new ideas for improvement, allowing for focused conversation.

To base our discussions on data-driven insights, I suggested reviewing sprint metrics such as velocity or burndown charts, which provided a clearer perspective on our workflow. This approach was particularly impactful in analyzing how we could improve our efficiency during projects like GOS Migration.

Finally, I concluded each retrospective by co-creating actionable items based on our discussions. By allocating responsibility for these follow-up actions, we ensured accountability and a commitment to improving our processes in the next sprint.

10. How do you adapt your communication style when discussing progress with stakeholders in an Agile environment?

Ans: Adapting my communication style to effectively engage stakeholders is paramount in an Agile environment. In my experience, particularly during projects like Security Remediation and Skyplan, I recognized that stakeholders have diverse levels of technical understanding and varying interests.

For technical stakeholders, I delve into details such as the architecture of our Spring Boot application and the complexities of our microservices development, providing updates on performance metrics and development timelines. This fosters trust and demonstrates that technical decisions are well-informed.

Conversely, for non-technical stakeholders, I focus on the business implications of our updates, using non-technical language to ensure clarity. For instance, instead of diving into the technical specifics of a JUnit test I developed, I would emphasize how that contributes to reduced errors and boosts efficiency, thereby ensuring the product's reliability.

Additionally, during our sprint reviews, I encouraged interactive sessions where stakeholders could see the software in action, which elicited real-time feedback. This engagement not only illustrates our progress but also encourages a collaborative environment.

In summary, I prioritize the context and expectations of my stakeholders, adapting the depth and tone of my communication accordingly to foster clarity

and collaboration.

11. What challenges did you face while utilizing JUnit and Mockito for testing in your projects, and how did you overcome them?

Ans: In the Skyplan project for United Airlines, I faced initial difficulties in achieving optimal test coverage with JUnit and Mockito.

Mocking complex dependencies created complications; however, I adopted a strategy of isolating components for testing, which allowed me to simulate interactions and behaviours effectively.

I also worked closely with business logic to identify critical paths and wrote comprehensive test cases, focusing on edge cases, which improved reliability. My efforts led to achieving over 85% test coverage, ensuring a robust application.

12. How do you ensure continuous collaboration among team members in an Agile development environment?

Ans: In my experience, especially at Mphasis where collaboration was essential for success, continuous collaboration among team members was achieved through regular meetings and open communication channels. Organizing daily stand-ups became vital in maintaining momentum and fostering collaboration.

During these meetings, each team member briefed their contributions and highlighted any impediments they're facing, which brought about quick resolutions as the entire team was engaged in overcoming challenges together.

I also emphasize collaboration tools like Jira and Slack to keep the dialogue ongoing, enabling team members to share updates, insights, and ideas throughout the day. This culture of collaboration helped us maintain a strong synergy. For example, while developing the Security Remediation project, consistent communication allowed the team to effectively address and resolve security vulnerabilities swiftly, highlighting the collective contributions of everyone. Furthermore, I ensured there was a culture of knowledge sharing. I organized knowledge transfer sessions where team members could present their findings and lessons learned from their tasks, including sharing coding experiences with Spring MVC or JDBC. This enabled the team to build collective expertise.

By creating an environment that emphasizes transparency and teamwork, we ensured that collaboration remained constant, ultimately resulting in heightened efficiency and a stronger product.

13. What measures do you take to secure Spring MVC applications throughout the MVC architecture?

Ans: Securing my Spring MVC applications is a top priority, and I implement multiple measures across the MVC architecture to protect user data and prevent unauthorized access.

One of the first steps is to utilize Spring Security to manage user authentication and authorization. This allows me to define access control on specific endpoints. For example, in the Skyplan project, I set up role-based access control to restrict certain functionalities to authenticated users only. Input validation is another crucial measure. I always validate and sanitize user inputs to prevent common security vulnerabilities like SQL injection and cross-site scripting (XSS). This practice is integral to ensuring the robustness of applications, especially when interacting with user-generated content.

Additionally, I enforce HTTPS across my applications to secure data transmission. Ensuring that all data sent between the server and client is encrypted helps safeguard sensitive information.

In the RTE project within Mphasis, I integrated CSRF protection to guard against cross-site request forgery attacks. Implementing these security protocols helped in maintaining the integrity of user sessions.

14. How do you incorporate test-driven development in an Agile setup, particularly with JUnit and Mockito?

Ans: Having designed and developed numerous projects like Skyplan and GOS Migration, I firmly believe that incorporating test-driven development (TDD) is crucial in an Agile environment. TDD allows us to write tests before the actual code, ensuring that code works as intended and significantly enhances the robustness of our application.

In my recent projects, I utilized JUnit alongside Mockito for unit testing. After defining our user stories during sprint planning, writing the tests became an integral part of the development cycle. As I developed, I would frequently run the tests, ensuring each feature performed correctly. For instance, while working on Skyplan, I created JUnit test cases that not only checked the behavior of specific Spring Boot services, but also utilized Mockito to mock dependencies effectively. This practice was particularly useful when dealing with GRPC services as it allowed us to simulate various scenarios without the need for a complete system. This method of testing accelerated our development time since we were able to detect issues early on, allowing the team to pivot if the approach we had taken wasn't optimal. The integration added significant value, resulting in over 85% test coverage for this project. By adhering to TDD principles, our Agile team achieved higher quality and maintainable code, ultimately making the transition to production smoother and more efficient.

15. What security considerations did you implement when developing APIs for projects like Oday?

Ans: Security was a paramount concern during the development of APIs in projects such as Oday. Given the sensitivity of the data handled, I adhered stringently to security best practices throughout the development cycle.

Firstly, I enforced OAuth2 standards for authorization, ensuring that only authenticated users could access sensitive endpoints. This included implementing secure token-based authentication, which protected against cross-site scripting and other vulnerabilities.

Additionally, I integrated input validation methods to prevent SQL injection attacks and utilized libraries that sanitize user input before processing it through the APIs.

Regular security audits and the use of tools like Veracode were integral to identifying and resolving vulnerabilities early in development. I actively participated in the code review process, focusing on potential security pitfalls.

To further enhance security post-deployment, I implemented logging and monitoring practices using platforms like Sonar to track any unusual activity and respond proactively to potential threats.

16. Can you explain the role of Spring MVC in your projects, specifically in terms of Controller implementation?

Ans: In my experience with Spring MVC, I've primarily utilized it to structure the web applications effectively by separating the application into different layers.

The Controller in Spring MVC serves as an intermediary between the model and the view. It accepts user input, processes it (potentially involving interaction with the model), and returns the appropriate view.

For instance, in my project with Mphasis (Wells Fargo) on the Bill Pay application, I created new REST controllers using Spring MVC. This allowed the application to handle multiple client requests while ensuring that the business logic remained encapsulated within specific service layers.

I implemented various controller annotations provided by Spring MVC, like `@Controller` and `@RequestMapping`, to define the endpoints for user access. It allowed for a clean and organized way to map URLs to specific actions within my application.

For instance, in the Oday project with Mphasis (FedEx), I utilized the Controller to direct traffic to the appropriate services, ensuring that the necessary data was fetched and processed correctly based on user interactions. This model-view-controller architecture also offered scalability, which was critical in migrating mainframe applications to Java applications using Spring Boot. The separation of concerns made the application easy to maintain and extend.

17. How do you manage versioning with REST APIs?

Ans: Managing versioning in REST APIs is vital for maintaining backward compatibility. I usually adopt a URI versioning strategy where the version

number is part of the API path, like /v1/resource.

This allows clients to specify which version they are accessing, enabling us to introduce improvements without breaking existing integrations.

I also ensure that the API documentation reflects the different versions and their respective capabilities. This transparency is crucial for client applications to adapt seamlessly.

By maintaining all versions of the API alongside each other, I can deprecate older versions gradually, giving clients enough time to transition.

18. Can you share an example of how you motivated your team to embrace Agile methodologies?

Ans: Motivating my team to embrace Agile methodologies has often been a critical part of my role as a developer. For instance, during my project on Oday at Mphasis, I realized the team was hesitant to depart from traditional methods. To address this, I first led by example, adhering to Agile principles in our workflow. I shared success stories from previous projects like Skyplan where implementing Agile led to noticeable improvements in timelines and product quality. When the team saw tangible results from Agile practices, it sparked curiosity and willingness to adopt these methodologies.

I implemented Agile workshops and training sessions that offered hands-on experiences, illustrating the real benefits of Agile processes and demonstrating tools like Jira for transparent project management.

Establishing a collaborative environment further reinforced the Agile mindset, as it allowed each team member to contribute their ideas and voice their concerns. I began implementing bi-weekly team challenges that aligned with Agile values, showcasing how collaboration could yield outstanding results.

By fostering an environment of trust, displaying the effectiveness of Agile processes through results, and facilitating opportunities for learning, my team embraced Agile methodologies wholeheartedly, yielding excellent outcomes in our projects.

19. Can you explain how your experience with Spring Boot and GRPC services at United Airlines has influenced your approach to microservices architecture?

Ans: During my tenure at United Airlines, I extensively worked on microservices architecture involving over 75 modules as part of the Security Remediation and Skyplan projects. My role primarily focused on creating and maintaining microservices using Spring Boot integrated with gRPC services.

The exposure to gRPC services taught me the importance of efficient communication between microservices especially in high-throughput and low-latency scenarios, such as in flight planning and operations at United Airlines. Designing microservices with a contract-first approach through protocol buffers ensured strong typing and backward compatibility, which is critical for seamless system upgrades.

Utilizing Spring Boot helped in rapid development by providing auto-configuration, embedded servers, and ease of dependency management via Maven and Gradle. This accelerated development while ensuring maintainability. I also leveraged containerization with Docker and CI/CD pipelines via Jenkins for automated deployment and smooth environment transitions.

From security remediation standpoint, integrating Veracode scans in build pipelines sharpened my skills in secure coding standards and vulnerability management, influencing me to architect microservices that comply with rigorous security audits and maintain high code quality with tools like Sonar.

In summary, these experiences shaped my holistic approach to microservices developmentâfocusing on scalable, secure, and well-tested services using technologies like Spring Boot, gRPC, Docker, and CI/CD pipelines essential in enterprise-grade applications.

20. What is your approach when documenting REST APIs?

Ans: Documenting REST APIs is essential for maintainability and developer onboarding. I primarily utilize Swagger in my projects, which auto-generates interactive documentation from the API annotations.

This documentation includes details about request parameters, response formats, and error codes. It provides a clear overview for both backend and frontend developers.

Moreover, I ensure that every newly added endpoint is documented alongside its implementation, thus reducing the risk of outdated information. Additionally, I may take the time to elaborate on complex endpoints in our internal wiki, which serves as a learning resource for the team.

21. What role do you believe documentation plays in database design and maintenance?

Ans: Documentation is extremely vital in all aspects of software development, especially in database design. In my role as a developer at Mphasis, I emphasized keeping clear and comprehensive documentation. This included detailing database schemas, entities, relationships, and constraints. Proper documentation ensures that other developers can quickly understand the design rationale and navigate the database structure effectively. Additionally, maintenance becomes significantly simpler when documentation outlines the expected data flows and object relationships, especially for long-term projects. It aids in onboarding new team members and supports project sustainability.

22. What role does feedback play in your Agile projects, and how do you ensure it is incorporated effectively?

Ans: In my experience, feedback is fundamental to the success of Agile projects. Particularly through my work on projects like BillPay and Taco Bell, I have learned that timely and structured feedback can significantly improve project trajectories.

During our sprint reviews, stakeholder feedback is actively sought. After each sprint, we conduct demonstrations, allowing stakeholders to interact with our product and provide real-time insights. Their feedback directly influences decision-making for the next sprint.

Furthermore, I promote a culture of peer reviews within the team. Encouraging team members to provide feedback on each other's code not only enhances code quality but fosters learning and collaboration among our developers. This process also highlighted the importance of collective ownership of our product. Moreover, incorporating feedback into our daily routines, such as during stand-ups and retrospectives, has shown us areas for improvement in our processes. Our retrospective discussions often brought up actionable insights which we documented and prioritized for future sprints.

Ultimately, effective incorporation of feedback reflects our commitment to continuous improvement, nurturing an iterative cycle that drives project excellence.

23. How do you ensure test coverage in your Spring MVC applications, particularly in your JUnit tests?

Ans: Ensuring high test coverage is crucial when working with Spring MVC applications. I emphasize writing comprehensive JUnit tests to validate the functionality of the controllers, services, and repositories.

In my most recent projects, such as the Skyplan project at United Airlines, I integrated testing early in the development process. By using Mockito for mocking dependencies, I was able to isolate the units of work effectively, ensuring that the tests were not dependent on the actual implementations of the services.

I implemented JUnit Test cases that exceeded 85% coverage by focusing on critical functionality paths and edge cases. I observed that thorough unit testing reduced the number of bugs and improved the refactoring process significantly.

In addition, I employed code coverage tools to identify untested paths within the application code, allowing me to focus my efforts on under-tested areas. It helped in providing confidence when making changes to the code, particularly during migrations from Java 1.8 to Java 17.

Moreover, I encouraged the implementation of a continuous integration (CI) pipeline to automate the testing process, ensuring that tests are executed each time a build is created. In the GOS Migration project at Mphasis, this CI approach was vital to maintain code quality.

24. How have you utilized JUnit and Mockito to achieve high test coverage in

backend development projects at Mphasis?

Ans: Throughout my projects, including Skyplan and Oday at Mphasis, I consistently emphasized writing robust unit and integration tests using JUnit and Mockito frameworks to maintain test coverage greater than 85%.

I created thorough unit tests for REST controllers, service layers, and data access layers to validate business logic and data flows. Mockito helped me mock external dependencies and isolate units under test, which was crucial for testing microservices independently without requiring full integration.

For instance, while developing gRPC services in Skyplan, I employed Mockito to simulate gRPC client and server interactions, ensuring message contracts and error handling were well tested under various scenarios, such as timeouts or invalid inputs.

Additionally, I leveraged parameterized testing in JUnit to cover multiple input combinations, edge cases, and exception paths, which improved test robustness further.

Integrating these tests into Jenkins pipelines enabled automated execution on every build, catching regressions early and contributing to stable continuous deployments in production environments.

This disciplined testing approach facilitated faster feedback loops, improved code quality, and increased confidence in complex backend systems handling mission-critical functionalities, like flight planning and payments processing.

24. Could you elaborate on your involvement with Veracode and other security scanning tools during your work on Security Remediation at United Airlines?

Ans: In the Security Remediation project at United Airlines, I was extensively involved in integrating Veracode static and dynamic code analysis into the build pipelines using Harness and Jenkins. This integration enabled automatic scanning of over 75 microservice modules for vulnerabilities and allowed us to catch security flaws early in the development cycle.

My responsibilities included interpreting Veracode scan results to identify and fix issues related to software component analysis (SCA), static code vulnerabilities, and dynamic testing findings obtained via tools like Invicti and NetSparker.

I reviewed codebases for security weaknesses such as injection flaws, weak cryptographic practices, and unsafe component usages, then refactored or patched the code accordingly. This also involved tightening dependency versions and auditing third-party libraries for known CVEs.

Furthermore, I actively participated in code reviews emphasizing secure coding practicesâone key aspect for meeting compliance and governance standards in aviation software.

By automating these security verifications and remediation workflows, the project minimized manual effort, reduced security debt, and led to delivering resilient, compliant applications that protect critical flight operations data. This role deepened my expertise in secure software development lifecycle (SSDLC) practices and gave me hands-on experience with prominent security scanning and remediation tools within large-scale enterprise environments.

25. How does Spring MVC support internationalization (i18n) in your applications?

Ans: Spring MVC offers robust support for internationalization (i18n), allowing my applications to cater to a global audience. I typically use message resource bundles to externalize messages and texts in various languages.

For instance, in a project where user interaction spans multiple regions, I create different properties files for each locale. For example, 'messages_en.properties' and 'messages_fr.properties' for English and French respectively. I then use MessageSource to load the correct resource bundle based on the user's locale.

I utilize the LocaleResolver to determine the user's locale, either from HTTP request headers or session attributes, allowing my application to respond accordingly. In projects like the Taco Bell learning portal, this feature ensured that users could access materials in their preferred language, enhancing usability.

Additionally, in the GOS Migration project at Mphasis, implementing i18n was crucial as the application was intended for international clients. Language-

specific messages enhanced user satisfaction and expanded our market reach. The integration of i18n in Spring MVC has proven to streamline the development process. By leveraging frameworks and features provided by Spring, I can focus on delivering effective business solutions while ensuring my applications are user-friendly across different languages.

26. How did you ensure high test coverage in your API projects, particularly in Skyplan and BillPay?

Ans: In both the Skyplan and BillPay projects, achieving high test coverage was a crucial goal for me. I focused extensively on writing unit tests and integration tests to validate each component of the API.

For unit testing, I used JUnit along with Mockito to create mocks of dependencies. This approach allowed me to isolate functionalities and validate them without relying on external services.

Additionally, I collaborated with my team to establish a testing strategy that included code reviews focusing on test scenarios. We consistently aimed for a minimum of 85% code coverage, which was achieved through rigorous testing practices.

We also paid attention to edge cases and failure scenarios, ensuring that our API would handle unexpected inputs gracefully. This proactive measure significantly reduced runtime errors post-deployment.

Adopting a test-driven development (TDD) approach in the early stages of the Skyplan project meant writing tests before the actual code. This ensured each feature was thoroughly vetted before integration, leading to a clean, efficient codebase.

27. Describe your experience with migration from Java 1.8 to Java 17 and the challenges you faced during your projects at Mphasis.

Ans: While working on the Security Remediation project for United Airlines through Mphasis, I participated in the migration of legacy codebases from Java 1.8 to Java 17. This upgrade involved understanding the new language features, module system, and the enhanced JVM capabilities.

One significant challenge was ensuring backward compatibility of existing microservices, especially since many of these were tightly coupled and had dependencies not yet compatible with Java 17. I conducted multiple proof-of-concepts (POCs) to verify code behavior, validate third-party libraries, and identify deprecated or removed APIs that needed refactoring.

Performance tuning was another focus area, where the new Garbage Collection algorithms and improvements in Java 17 helped reduce latency and resource consumption, essential for high-throughput applications such as those in the aviation domain.

I also focused on leveraging new language features such as sealed classes and improved pattern matching that enhanced code readability and maintainability across services.

The biggest learning was automating the migration process by incorporating static code analysis tools and comprehensive unit testing using frameworks like JUnit and Mockito to maintain code coverage above 85%, which ensured system stability during and after migration.

This experience enriched my skills in modernizing enterprise Java applications while balancing risk and performance improvements.

28. How do you approach designing user-friendly interfaces while using Spring MVC?

Ans: Designing user-friendly interfaces with Spring MVC is crucial for enhancing user experience, and I approach this by focusing on clean architecture and effective design practices.

Initially, I ensure that the web application layout is intuitive. I spend time collaborating with UI/UX designers to create mockups that are straightforward and aesthetically appealing. During my time working on the Taco Bell learning portal, we iterated designs based on user feedback to achieve maximum usability. I also leverage Thymeleaf as a templating engine, which works seamlessly with Spring MVC. This allows dynamic content to be rendered effectively while providing a familiar syntax for developers. Using Thymeleaf greatly enhances the user interaction experience by enabling dynamic rendering based on specific user

actions.

It's also essential to consider response times and content delivery. I utilize AJAX calls to load parts of the web pages asynchronously, allowing users to interact with the application without page reloads. This technique was particularly useful in implementing features in the Payment Connect project where seamless customer interactions were a priority.

Lastly, I continuously gather user feedback and analytics to iteratively improve the interface. Adapting the design based on real user experiences ensures that the final product meets the intended usage effectively.

29. In what scenarios do you prefer using RESTful web services within Spring MVC?

Ans: I prefer using RESTful web services within Spring MVC in scenarios where an application requires stateless interactions over the HTTP protocol. This approach aligns well with the microservices architecture I often implement, such as in the Skyplan project at United Airlines.

In microservices, different services communicate with each other through well-defined APIs. Using RESTful services allows me to leverage the HTTP methods (GET, POST, PUT, DELETE) efficiently, making architectural decisions clearer and more straightforward.

Moreover, RESTful web services are inherently scalable and facilitate the development of loosely coupled systems. Given my experience in migrating applications to the cloud at Mphasis, it was crucial to ensure that services could scale independently based on traffic requirements.

For example, in my work with the Payment Connect application, I designed services that catered specifically to payment transactions. The flexibility of REST allowed me to develop new endpoints that can be versioned easily, ensuring backward compatibility.

Another advantage is the ease of integrating with front-end frameworks or mobile applications. Spring MVC allows me to develop RESTful APIs that return data in a standardized format like JSON, which is immensely beneficial when designing responsive front-end solutions.

30. How do you test your REST APIs and what frameworks do you use?

Ans: Testing is crucial for ensuring that my REST APIs perform as intended. I employ JUnit combined with Mockito for unit testing, which allows me to mock external dependencies and focus on testing the actual endpoint logic.

I also use Postman for manual testing of APIs, enabling me to simulate requests and validate responses in a user-friendly interface.

Furthermore, I implement integration tests that validate the interaction between different layers of the application, ensuring that data flows correctly through the entire stack.

This comprehensive testing strategy has helped maintain high code quality and reliability across all RESTful Web Services.

31. How did you handle database migrations in your projects, particularly with GOS Migration?

Ans: For the GOS Migration project, managing transitions smoothly was a major focus. We used Liquibase for managing the database migrations, allowing for version control of the database itself, similar to how we would with application code.

This strategy allowed us to apply schema changes in a controlled manner. Each change was thoroughly tested in a staging environment before being applied to production, minimizing the risk of downtime.

Additionally, I evolved the deployment scripts to cater for rollbacks in case of any issues after migration. This ensured that we could revert to the previous working state without complications.

32. What challenges have you faced while applying Agile practices in your previous roles, and how did you overcome them?

Ans: Throughout my career, particularly while working on extensive projects within large teams at Mphasis, I've encountered several challenges while implementing Agile practices. A notable challenge was resistance to change; many team members were accustomed to traditional methodologies, leading to conflicts

in adoption.

To tackle this, I facilitated workshops to educate the team on the benefits of Agile, illustrating case studies from projects such as Taco Bell and FX0 where Agile methodologies led to impressive enhancements in project efficiency and product quality.

Moreover, during my time with United Airlines, integrating diverse teams from different departments brought its own set of issues due to varying levels of expertise and understanding of Agile processes. I addressed this by ensuring tailored communication for each team, breaking down the complexities of Scrum and Kanban in a way that made it relatable for everyone involved.

Another common challenge was effectively managing stakeholder expectations during the iterative process. To overcome this, I made it a priority to involve stakeholders in sprint reviews, ensuring they were always informed and involved in updates. This transparency not only cultivated trust but also garnered valuable feedback for our teams.

In conclusion, these experiences have taught me the importance of adaptability and proactivity when implementing Agile practices, and the vital role of fostering open communication within teams to ensure a smooth transition towards a more efficient workflow.

33. How do you handle security incident response throughout your development lifecycle?

Ans: Handling security incident response effectively is essential, and I have built a systematic approach over the years.

During my projects in United Airlines, I established incident response protocols that include identifying incidents, assessing their severity, and containing them.

This involves setting up alerts for suspicious activities using logging and monitoring tools.

Collaboration with cross-functional teams is critical in this phase; we design a remediation plan for each incident and perform root cause analysis.

Lastly, documenting lessons learned helps in refining processes and ensuring better preparedness for future incidents.

34. Describe how you implemented secure deployment practices in your past roles?

Ans: In my previous positions, particularly with Mphasis, I worked to implement secure deployment practices across various projects.

One primary approach was utilizing Jenkins to automate deployment while integrating security checks in the pipeline.

This included ensuring that only images from trusted repositories were pulled, and using Docker best practices like running containers with non-root users. Additionally, I emphasized the use of tools that facilitate secret management within the deployment processes.

Monitoring running applications for vulnerabilities post-deployment also became part of our routine, ensuring ongoing compliance with the latest security standards.

35. How did you ensure the security of your REST APIs?

Security was a major concern for the REST APIs I developed. To mitigate risks, I implemented OAuth 2.0 for secure authentication, which provided a secure and standard approach for user authorization.

I also enforced HTTPS for all API requests to encrypt data in transit, thus securing sensitive information such as user credentials.

Moreover, I regularly conducted vulnerability assessments using Veracode scans integrated into our CI/CD pipeline to catch security flaws early in the development process.

By incorporating these measures, I was able to ensure that our APIs remained secure from common threats and attacks.

36. Can you discuss your approach to handling errors and exceptions in your API development?

Ans: Handling errors and exceptions effectively is crucial for providing a smooth user experience, and I have emphasized this across all my API projects, including Payment Connect and Oday.

In my implementation, I structured error handling by utilizing global exception handlers in Spring Boot. This allowed me to define consistent error response formats across all APIs, greatly enhancing clarity for API consumers. Next, I categorized errors into distinct types such as client errors (4xx) and server errors (5xx), ensuring that the appropriate HTTP status codes were returned for different scenarios. This granularity allowed clients to understand the nature of failures without needing to parse through extensive error logs. Furthermore, I implemented a comprehensive logging mechanism that captured error stack traces and relevant request details, which facilitated troubleshooting and debugging. I employed logs to monitor the health of the API and track recurring issues. Finally, I adopted a proactive approach to error reporting by providing meaningful messages in response bodies that guided users on how to mitigate issues, thus enhancing overall user satisfaction.

37. Can you describe a scenario where you implemented Docker in your microservices architecture?

Ans: In my role at Mphasis, while working on the GOS Migration project, I successfully implemented Docker for containerizing our microservices. This approach allowed us to maintain consistency across various development environments.

I created Docker images for each microservice, ensuring that we could deploy them seamlessly into our CI/CD pipeline.

Through Docker, I could manage dependencies effectively, resulting in improved collaboration between development and operations teams, as the environment was predictable.

38. What performance optimization techniques have you implemented for your RESTful Web Services?

Ans: To optimize performance for RESTful Web Services, I utilize several techniques. Caching is one of the first strategies I implement, leveraging Redis to store response data that does not change frequently. This significantly reduces latency for frequently accessed resources.

I also analyze request payload sizes and optimize them by only returning necessary data fields through the use of DTOs (Data Transfer Objects).

Additionally, I utilize HTTP/2 where possible to benefit from multiplexed requests and server push capabilities, improving the speed of data transmission. Implementing pagination for large datasets is another crucial measure, ensuring that clients only retrieve manageable amounts of data at once.

39. Can you discuss your approach to optimizing database queries in your projects like Oday?

Ans: In the Oday project, optimization was critical, particularly due to the large volume of data processed in migrating the mainframe application. My approach began with analyzing the existing queries for efficiency and identifying slow queries using tools like the SQL execution plan.

Once I identified the bottlenecks, I optimized join operations between tables by ensuring that proper indexing was in place. For example, I used composite indexes on commonly queried fields to speed up query execution times.

I also refactored certain queries to reduce complexity and improved the overall design of the data access layer to minimize round-trips for data retrieval. Over time, these optimizations led to noticeable performance improvements and greater user satisfaction.

40. What role did API documentation play in your projects, like Security Remediation?

Ans: In the Security Remediation project, comprehensive API documentation was essential for several reasons. First, it facilitated communication among team members, ensuring that everyone was aligned on the expected behavior of the APIs.

By using Swagger for documentation, I could easily generate interactive API docs that allowed developers to test endpoints directly. This tool not only improved usability for the API consumers but also served as an effective onboarding tool for new team members.

The documentation provided clear usage examples, detailed parameter descriptions, and indicated expected responses. This clarity reduced the number of support queries from internal teams and clients, streamlining the development process.

Furthermore, as we integrated additional security features, maintaining up-to-date documentation became critical. It was essential for ensuring that both existing and new consumers understood any changes in API behavior or newly required parameters for requests.

Overall, prioritizing documentation made the development more fruitful and efficient, as less time was wasted on clarifications and troubleshooting.

41.How do you handle conflicts within an Agile team, and what strategies do you employ to resolve them?

Ans: Handling conflicts within an Agile team is a critical skill. Throughout my career, notably while working on collaborative projects at Mphasis, I faced conflicts that arose from various sources such as differing opinions or workload pressures.

My first approach is to foster open communication. I encourage team members to voice their concerns, ensuring that all voices are heard during meetings. For example, during a particularly challenging phase in the GOS Migration project, differing opinions on workflow priorities among team members led to tension. To resolve this, I organized a conflict resolution meeting, providing a platform for all team members to express their viewpoints constructively. I facilitated these discussions to ensure they were respectful and productive, which allowed us to see the situation from multiple perspectives and build mutual understanding.

Another strategy I employ is to focus on the common goal that unites the team. Reminding team members of the project's objectives fosters a collective mindset rather than individual viewpoints, which tends to refocus discussions constructively.

Moreover, I advocate for utilizing collaborative tools, like shared documentation, where members can express their suggestions or concerns. This transparency not only mitigates misunderstandings but also serves as a reference point during resolution processes.

Overall, my focus on open communication, collaborative problem-solving, and emphasizing common goals allows me to effectively manage and resolve conflicts within Agile teams.

footer-thumbs-upfooter-thumbs-down

42.How did your academic background from Noor College of Engineering and Jawaharlal Nehru Technological University prepare you for practical challenges in backend development?

Ans: Graduating with a B.Tech in Computer Science & Engineering from Noor College of Engineering, affiliated to Jawaharlal Nehru Technological University, Hyderabad, laid a solid theoretical foundation in data structures, algorithms, database management, and software engineering principles.

The curriculum introduced me to core subjects like operating systems, networking, and object-oriented programming, which underpinned my understanding of backend system design and performance considerations.

I engaged in academic projects that involved basic web and database application development, which sparked my interest in Java technologies and later led me to specialize in Java Enterprise Edition, including frameworks like Spring and Hibernate.

The university also emphasized analytical thinking and problem-solving skills, which I applied throughout my career in solving complex business logic, architectural design, and debugging issues in large-scale enterprise systems. The exposure to programming fundamentals and engineering methodology during college gave me the discipline needed to adapt quickly to evolving backend technologies such as microservices, containerization, and cloud deployments. Thus, my academic journey was instrumental in equipping me with the core competencies and mindset required for success in backend development roles at reputed organizations like Mphasis.

footer-thumbs-upfooter-thumbs-down

43. What technologies did you integrate with Spring ecosystem in your applications to enhance functionality?

Ans: In the Taco Bell project, I integrated various technologies with the Spring ecosystem.

I utilized Hibernate for ORM, which worked seamlessly with Spring to manage data persistence.

For message brokering, I incorporated JMS, which significantly improved our system's responsiveness.

Moreover, leveraging Swagger allowed me to document and visualize the RESTful APIs, enhancing collaboration with front-end developers.

footer-thumbs-upfooter-thumbs-down

44. Can you explain the role of Spring MVC in your RESTful Web Services?

Ans: Spring MVC played a pivotal role in structuring my RESTful Web Services. It provided a clear separation of concerns by allowing me to separate the controller, service, and repository layers effectively.

Using the `@RestController` annotation, I was able to create controllers specifically for handling API requests, simplifying the implementation of REST endpoints.

Additionally, Spring MVC facilitated request mapping through `@RequestMapping` and `@GetMapping` annotations, making it easier to define and manage routes for different HTTP methods.

Overall, Spring MVC allowed me to maintain a clean and organized codebase while rapidly developing RESTful APIs.

footer-thumbs-upfooter-thumbs-down

45. Can you discuss how Spring MVC helps with the separation of concerns in your projects?

Ans: Absolutely! The separation of concerns is a fundamental principle I adhere to in my projects, and Spring MVC facilitates this through its well-defined architecture.

With Spring MVC, I can distinctively separate the presentation layer, business logic, and data access layer. This separation simplifies maintenance and enhances the scalability of my applications.

For instance, when developing the Bill Pay application at Mphasis, I isolated the control flow by utilizing Controllers to handle user requests. The business logic resides in Service classes, and Repositories manage data access. This clear division allows different team members to work on independent components without affecting the entire application.

Moreover, this architecture supports better testing practices. Each layer can be tested in isolation, making unit tests more effective. For example, while working on the Oday project, I was able to use Mockito effectively to mock dependencies and focus solely on testing the controller logic.

This separation also enables better code organization, where different packages can hold files related to their respective functionality, enhancing readability.

Finally, Spring MVC promotes a more modular approach, allowing for code reuse and easier integration of new features in ongoing projects.

footer-thumbs-upfooter-thumbs-down

46. Can you discuss your experience with creating APIs using Spring Boot in your projects like Skyplan?

Ans: In my role at United Airlines, I developed APIs as part of the Skyplan project. Utilizing Spring Boot, I implemented the GRPC service architecture to facilitate robust and efficient communication between microservices.

The primary goal was to enable seamless interaction among flight planning modules. I created specific endpoints to handle requests for flight plans, coordinating data from multiple sources.

To ensure reliability and maintainability, I followed best practices in RESTful API design, including proper status codes and response messages. This structure allowed for easier consumption of the API by front-end applications and third-party services.

Additionally, I employed JWT tokens for authentication in API calls, enhancing security during data transmission between clients and servers.

Moreover, integrating API documentation with tools like Swagger was critical to making the API self-explanatory and facilitated onboarding new developers and clients.

footer-thumbs-upfooter-thumbs-down

47. What has been your experience with integrating external APIs in your projects?

Ans: Integrating external APIs has been a significant part of my experience, particularly in projects like GOS Migration and Taco Bell. In Taco Bell, I developed a payroll system that required integrating with various external APIs for employee data validation.

In GOS Migration, I integrated external weather data APIs to enhance the flight-planning features of the service. This required handling real-time data effectively and ensuring that our system could process it asynchronously.

During integration, I ensured that all external interactions utilized robust error handling and retry mechanisms to deal with the unreliability of third-party services. This comprehensive error management minimized the ripple effect on the internal services and provided stable performance.

I also implemented caching strategies for responses received from these external services to reduce load times and optimize performance effectively.

All in all, these experiences enabled me to reinforce my adaptability and develop skills in working with varying data formats and protocols that external APIs may offer.